

Chương 1

Lập trình C++ cơ bản (phần 2)

Nội dung

- 1.1 Chương trình C++
- 1.2 Kiểu dữ liệu, biểu thức và phép toán
- 1.3 Lệnh điều khiển
- 1.4 Kiểu dữ liệu mảng
- 1.5 Kiểu dữ liệu cấu trúc
- 1.6 Hàm

1.4 Kiểu dữ liệu mảng

- **Mảng:** là dãy gồm nhiều phần tử nối tiếp nhau, mỗi phần tử là một **biến dữ liệu cùng kiểu**
- Mỗi phần tử của mảng được định danh qua **tên mảng** và ***chỉ số phần tử***
- Cấu trúc của mảng xác định qua số chiều

A) Mảng 1 chiều

- Khai báo gồm: kiểu dữ liệu của phần tử, tên mảng và số lượng phần tử

```
int A[10];    // A là tên mảng
```

```
char s[100];
```

```
float dayso[10];
```

- Chú ý: số lượng phần tử phải là hằng hoặc biểu thức hằng

```
const size_t SIZE = 10;
```

```
int B[SIZE];
```

```
int C[SIZE+10];
```

Thao tác với phần tử mảng

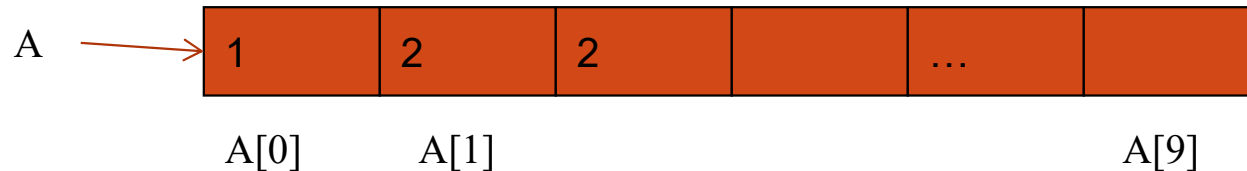
- Truy xuất đến phần tử qua chỉ số

```
i = A[0] = 1;
```

$$A[i] = A[i+1] = 2$$

- Chú ý: chỉ số mảng bắt đầu từ 0

- Chỉ số phải là số nguyên không âm và nhỏ hơn số lượng phần tử: $0 \leq i < n$ (n là số phần tử)



B) Mảng 2 chiều

- Khai báo gồm: kiểu dữ liệu, tên mảng, kích thước mảng theo từng chiều

`int A[10][10]; // số phần tử = 10 x 10`

- Mảng 2 chiều được sử dụng để lưu dữ liệu
 - Dạng bảng 2 chiều (hàng và cột)
 - Dạng ma trận
 - Nhiều mảng 1 chiều cùng kích thước
 - Dạng lưới chia ô trên mặt phẳng 2 chiều

1 2 4
 5 6 8
 2 3 4

Ma trận

	Thắng	Hòa	Thua	Điểm
AC Milan	1	1	1	4
MU	3	0	0	9
Ajax	1	1	1	4
Auxerre	0	0	3	0

Bảng

↑	↓	→
→	↑	↗
↓	→	?

Lưới 2 chiều

Cách nhập và in toàn bộ phần tử

- Sử dụng 2 cấu trúc for lồng nhau

- Nhập dữ liệu phần tử

```
for(i=0;i<n;i++)  
    for(j=0;j<m;j++)  
    {  
        cout << "A[" << i << "," << j << "]=";  
        cin  >> A[i][j];  
    }
```

- In giá trị phần tử

```
for(i=0;i<n;i++)  
{  
    for(j=0;j<m;j++)  
        cout << setw(5) << A[i][j]);  
    cout << endl;; // xuống dòng  
}
```

Kiểu chuỗi ký tự cơ bản

- Chuỗi ký tự cơ bản là mảng 1 chiều với phần tử là ký tự
`char ten[100] = "Tran Van Tuan";`
`char lop[40] = "63IT1";`
- In chuỗi ký tự
`cout << ten << " lop" << lop << endl;`
- Nhập chuỗi ký tự
`cin.getline(ten, 100); // độ dài tối đa 100 ký tự`
- Truy nhập ký tự
`cout << ten[0] ; // ký tự T`

Bài tập minh họa

1.5) Kiểu dữ liệu cấu trúc

- Kiểu struct: biểu diễn dữ liệu **phức hợp** gồm nhiều trường dữ liệu, do người lập trình tự chọn.
- Kiểu struct dùng để mô tả các dạng dữ liệu mà mỗi đại lượng dữ liệu gồm nhiều thành phần thông tin

Ví dụ: mỗi 1 hồ sơ sinh viên gồm các thành phần:

- Họ tên, giới tính, ngày sinh, quê quán, địa chỉ, số CMTND, mã số SV, lớp, năm nhập trường ...

1.5) Kiểu dữ liệu cấu trúc

- Struct giúp tổ chức, liên kết các thành phần dữ liệu liên quan vào 1 đại lượng dữ liệu chung
- Không dùng struct, vẫn có thể biểu diễn được dữ liệu nhưng sẽ rời rạc và không có tính cấu trúc

VD: Ma trận thường được biểu diễn với các dữ liệu sau

```
unsigned int  n,m; // số hàng và cột  
int  A[10][10];  // các phần tử
```

Tuy nhiên, các dữ liệu này đơn thuần là 2 số nguyên và 1 mảng 2 chiều độc lập, không có mối liên kết trong cùng 1 đại lượng dữ liệu chung là ma trận

Khai báo (tạo) kiểu struct

- Sử dụng từ khóa struct, tên kiểu tự chọn và khai báo các thành phần dữ liệu
- Khai báo kiểu struct MaTran

```
struct MaTran
{
    int hang; // số hàng
    int cot;  // số cột
    int E[10][10]; // mảng chứa phần tử
};
```

Khai báo biến dữ liệu struct

- Sử dụng tên kiểu struct và tên biến

MaTran mA, mB, mC;

- Có thể kết hợp vừa khai báo kiểu và khai báo biến struct

```
struct MaTran
{
    int hang; // số hàng
    int cot;  // số cột
    int E[10][10]; // mảng chứa phần tử
} mA, mB, mC;
```

Thao tác với biến dữ liệu struct

- Nhập dữ liệu: sử dụng cú pháp <biến struct>.<tên thành phần>

```
MaTran mA;  
cout << "Nhap so hang:"; cin >> mA.hang;  
cout << "Nhap so cot:"; cin >> mA.cot;  
cout << "Nhap phan tu:" << endl;  
for(i=0;i<mA.hang;i++)  
    for(j=0;j<mA.cot;j++)  
    {  
        cout << "E[" << i << ", " << j << "]=";  
        cin >> mA.E[i][j];  
    }
```

- Tính toán và in ra màn hình cũng dùng cú pháp trên

Mảng với phần tử là struct

- Khai báo với tên kiểu struct, tên mảng và số phần tử

`MaTran dayMaTran[6]; // 6 ma trận`

- Truy nhập phần tử mảng và thành phần

Số hàng của ma trận chỉ số 0: `dayMaTran[0].n`

Số cột của ma trận chỉ số 3: `dayMaTran[3].m`

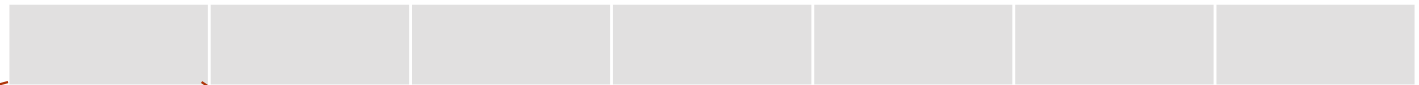
Phần tử i, j của ma trận chỉ số i : `dayMaTran[i].E[i][j]`

Ví dụ minh họa

```
MaTran dayMaTran[6];
```

dayMaTran[0]

dayMaTran[5]



$n = 2$
$m = 2$
$E[0][0] = 1$
$E[0][1] = 0$
$E[1][0] = 0$
$E[1][1] = 1$

Ma trận

dayMaTran[0]

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Định nghĩa tên kiểu dữ liệu với từ khóa typedef

- Người lập trình có thể đặt tên mới cho một kiểu dữ liệu tùy ý
- Cú pháp

typedef <tên kiểu đã có> <tên kiểu mới>;

```
typedef struct MaTran
{
    unsigned int n;
    unsigned int m;
    int E[10][10];
}MaTranSoNguyen;
```

```
// khai báo với tên kiểu mới
MaTranSoNguyen mA, mB, mC;
```

Bài tập minh họa

1.6) Hàm

- 1.6.1 Khái niệm
- 1.6.2 Cách tạo hàm trong C++
- 1.6.3 Lời gọi hàm

1.6.1 Khái niệm về hàm

- Phương pháp lập trình thủ tục là cách tổ chức chương trình thành nhiều mô đun liên kết với nhau
- Mỗi mô đun chương trình con thực hiện một công việc nhất định trong chương trình lớn
- Mô đun chương trình con trong C được gọi là HÀM (function)
- Chương trình C nói chung là kết hợp của
 - Hàm lập trình viên tự viết (user-defined function)
 - Hàm chuẩn trong thư viện của C
- Nhiều hàm có liên quan đến nhau về mặt tác vụ hoặc mục đích có thể nhóm lại trong cùng một mô đun cấp cao hơn. VD: thư viện `stdio`, `iostream`, `math`,...

1.6.2 Cách tạo HÀM C++

- Khai báo **nguyên mẫu hàm** (function prototype)
- Lập trình phần xử lý bên trong hàm (function definition)

A) Khai báo nguyên mẫu

- Cú pháp:

`kiểu_dl_trả_về      tên_hàm(danh_sách_tham_số) ;`

Ví dụ: nguyên mẫu hàm giá trị max của 2 số nguyên x, y

Cách khai báo 1: `int max(int, int) ;`

Cách khai báo 2: `int max(int x, int y) ;`

Tiếp

- Chú ý:
 - Danh sách tham số có thể trống (không tham số)
 - Kiểu trả về của hàm có thể là **void** (hàm không trả lại giá trị nào)

```
void    printHelloWorld();
```

B) Lập trình xử lý bên trong hàm (định nghĩa hàm)

- Cú pháp

```
<kiểu_dl> <tên_hàm> (<ds_tham_số_hình_thức>)  
{  
    // phần thân hàm (function body)  
    Khai_báo_biến_dl_cục_bộ;  
    Dãy_câu_lệnh_xử_lý;  
}
```

- Biến cục bộ là biến dữ liệu được sử dụng bên trong hàm
- Dãy câu lệnh thực hiện thao tác xử lý, tính toán của hàm

Chương trình minh họa

- Hàm tính tổng bình phương của 2 số thực

```
float tong_binh_phuong(float a, float b)
{
    float tong; // biến cục bộ
    tong = a*a + b*b;
    return tong;
}
```

Danh sách
tham số



Trả về kết quả



Tiếp

- *Chú ý:* lệnh `return`
 - Khi thực hiện lệnh `return`, hàm sẽ kết thúc và trả lại giá trị biểu thức đặt sau `return`
 - Nếu kiểu của hàm là `void`, sau `return` để trống:
`return ;`

Chương trình ví dụ

- Lập hàm tính giá trị lớn nhất của một dãy số nguyên đầu vào A[]
 - Xử lý: Duyệt dãy và tìm giá trị lớn nhất
 - Kiểu trả về: int
 - Tham số đầu vào
 - Mảng A phần tử nguyên
 - Số phần tử n
 - Giá trị trả về: phần tử lớn nhất trong mảng A

1.6.3 Lời gọi hàm

- Sau khi tạo hàm, để sử dụng cần phải **gọi hàm (call function)**
- Lời gọi hàm (function call) bao gồm
 - Tên hàm
 - Các tham số **truyền cho hàm**
- Ví dụ

```
int x, y, tong;
```

```
x = y = 5; // x và y là 2 tham số truyền cho hàm
```

```
tong = tong_binh_phuong(x, y);
```



Tên hàm

Tham số thực sự

Tham số hình thức và tham số thực sự

Tham số hình thức

- Đặt trong định nghĩa hàm

```
// a và b là tham số hình thức
float max(float a, float b)
{
    return a > b ? a : b ;
}
```

Tham số thực sự

- Đặt trong lời gọi hàm

```
// x và 5.5 là tham số thực
sự
int m, x = 7.7;
m = max(x, 5.5);
```

Chú ý

- Số lượng tham số thực sự = số lượng tham số hình thức
- Kiểu dữ liệu của tham số thực sự phải **tương thích** với kiểu của tham số hình thức

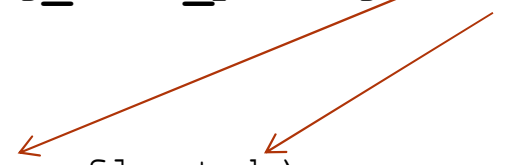
`m = max(&x, 5.5) // sai vì sử dụng tham số thực sự là địa chỉ`

B) Truyền tham số

- Truyền tham số là truyền tham số thực sự vào bên trong phần thân hàm thông qua tham số hình thức

```
int x = 3;  
tong = tong_binh_phuong(2, x);
```

```
float tong_binh_phuong(float a, float b)  
{  
    float tong; // biến cục bộ  
    tong = a*a + b*b;  
    return tong;  
}
```



Các kiểu truyền tham số

Truyền giá trị

- **Tham số hình thức là bản sao giá trị của tham số thực sự**
- Kết quả: thay đổi tham số hình thức không làm ảnh hưởng đến tham số thực sự

Truyền tham chiếu

- Tham số hình thức là một định danh khác (bí danh) của chính tham số thực sự
- Kết quả: thay đổi giá trị của tham số hình thức đồng nghĩa với thay đổi giá trị của tham số thực sự

Chương trình minh họa

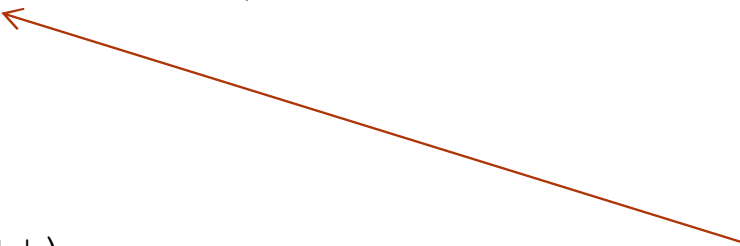
- Cho số nguyên a và b, hãy viết một hàm hoán đổi giá trị giữa a và b (hàm swap)
- Áp dụng 2 kiểu truyền tham số cho hàm nói trên

C) Tham số kiểu mảng

- Có thể đặt mảng 1 chiều, mảng 2 chiều làm tham số hình thức của hàm

```
void    maxArray(int    A[100],    int n)
{
    int max = A[0];
    int i;
    for(i=1; i<n; i++)
        if(max<A[i])    max = A[i];
    return max;
}
```

tham số
hình thức
là mảng 1
chiều



Chú ý

- Tham số thực sự là **địa chỉ của mảng có số chiều và kích thước tương ứng** với tham số hình thức

```
int B[100];  
// Nhập dữ liệu cho mảng B  
...  
// Tìm phần tử lớn nhất trong m phần tử của B  
int max_value = maxArray(B, m);
```

Chương trình minh họa

- Hàm void sapxep(int A[], int n) thực hiện việc sắp xếp một dãy số (mảng 1 chiều) theo thứ tự tăng dần.
- Tham số:
 - A là mảng 1 chiều cần sắp xếp
 - n là số phần tử trong mảng
- Cũng có thể truyền tham số dạng con trỏ như sau
 - void sapxep(int* A, int n)
 - A là con trỏ, trỏ đến mảng 1 chiều cần sắp xếp

1.6.4 Phạm vi sử dụng của biến

- **Phạm vi** (scope) của một biến dữ liệu: là phạm vi trong chương trình mà biến tồn tại và có thể truy cập
- Tùy theo phạm vi, một biến sẽ có **thời gian tồn tại và vị trí bộ nhớ** khác nhau
- **Thời gian tồn tại** là từ thời điểm biến được cấp bộ nhớ cho đến khi giải phóng khỏi bộ nhớ chương trình
- **Vị trí bộ nhớ**: vị trí vùng bộ nhớ cấp cho biến để lưu giá trị dữ liệu

Biến cục bộ và toàn cục

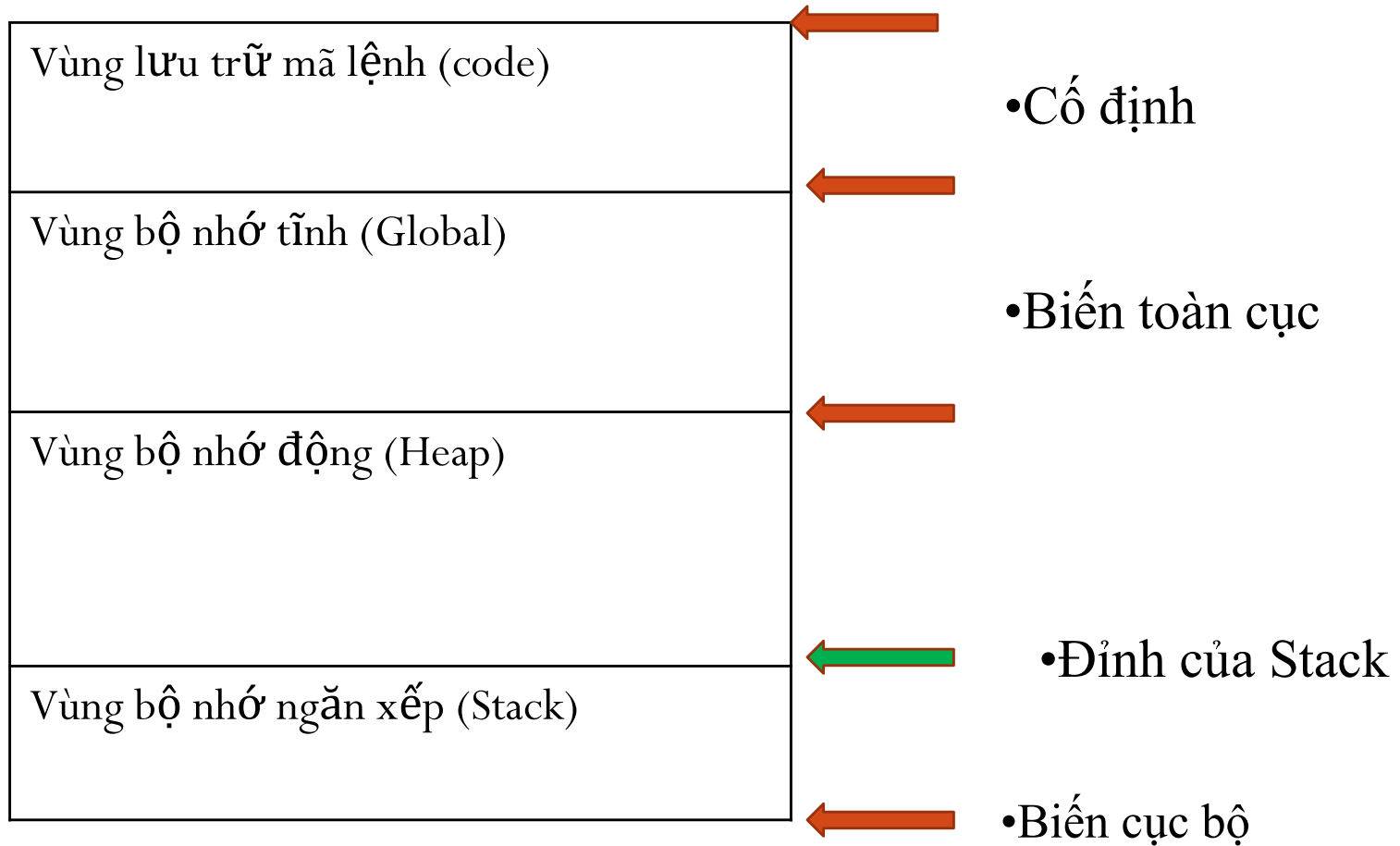
• Biến cục bộ

- Biến cục bộ là biến khai báo trong thân một hàm
- Phạm vi là bên trong hàm
- Tồn tại trong khoảng thời gian thực thi hàm
- Cấp phát tại vùng bộ nhớ Stack mỗi khi hàm được gọi

• Biến toàn cục

- Biến toàn cục là biến khai báo bên ngoài, không thuộc bất cứ hàm nào
- Phạm vi là trong toàn bộ chương trình
- Tồn tại trong suốt thời gian thực hiện chương trình
- Cấp phát tại vùng bộ nhớ Global ngay khi chương trình bắt đầu

Các vùng trong bộ nhớ của 1 chương trình máy tính



Biến cục bộ

- Nếu khai báo biến cục bộ nằm trong một khối lệnh thì biến chỉ có ý nghĩa trong phạm vi khối lệnh đó
- Khi thực thi xong khối lệnh thì biến được giải phóng và không còn tồn tại

Khối
lệnh

```
int i; // biến i ngoài khối
for(i=0; i<10; i++)
{ // khối lệnh
    int x // biến x nằm trong khối, cấp lưu trữ động
    x = i*2;
    printf("%4d", x);
}
x = 3; // Sai, báo lỗi
// ra ngoài khối, biến x không còn tác dụng
```

Ví dụ biến toàn cục và biến cục bộ

```
#include <stdio.h>
#include <conio.h>
int ketqua; // biến toàn cục
float tong_binh_phuong(float a, float b)
{
    float tong; // biến cục bộ của hàm
    tong = a*a + b*b;
    return tong;
}

int main()
{
    float x = 5.5, y = 6.6; // biến cục bộ hàm main
    ketqua = tong_binh_phuong(x, y);
    printf("Ket qua: %6.2f \n", ketqua);
    getch();
    return 0;
}
```